

AI ASSISTED CODING (ASSIGNMENT-11.1)

Name: V .Madhav Reddy

Roll No: 2503A52698

Batch: 24

#Q1

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Prompt: generate a code on Stack class with push, pop, peek, and is_empty methods.

Code:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.items = []
```

```
    def push(self, item):
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        if self.is_empty():
```

```
            return None
```

```
        return self.items.pop()
```

```
    def peek(self):
```

```

        if self.is_empty():
            return None

        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

stack = Stack()

stack.push(10)

stack.push(20)

print("Top element:", stack.peek())

print("Popped element:", stack.pop())

print("Is stack empty?", stack.is_empty())

```

Output:

```

PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/T11.py"
Top element: 20
Popped element: 20
Is stack empty? False
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#Q2

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Prompt: generate a code to implement a Queue using python list.

Code:

```
class Queue:
```

```
    def __init__(self):
```

```
    self._items = []

def enqueue(self, item):

    """Add an item to the rear of the queue."""

    self._items.append(item)

def dequeue(self):

    if self.is_empty():

        return None

    return self._items.pop(0)

def peek(self):

    if self.is_empty():

        return None

    return self._items[0]

def is_empty(self):

    return len(self._items) == 0

if __name__ == "__main__":

    queue = Queue()

    print("Is queue empty?", queue.is_empty())

    queue.enqueue(10)

    queue.enqueue(20)

    queue.enqueue(30)

    print("Queue after enqueueing 10, 20, 30:", queue._items)

    print("Front item:", queue.peek())

    print("Dequeued item:")

    print(queue.dequeue())

    print("Front item after dequeue:", queue.peek())
```

```
print("Is queue empty?", queue.is_empty())
```

output:

```
PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/t11.2.py"
Is queue empty? True
Queue after enqueueing 10, 20, 30: [10, 20, 30]
Front item: 10
Dequeued item:
10
Front item after dequeue: 20
Is queue empty? False
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

#Q3

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
pass
```

```
class LinkedList:
```

```
pass
```

Prompt: generate a code on singly linked list with insert and display methods.

Code:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insert(self, data):
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```

        self.head = new_node

    return

current = self.head

while current.next is not None:

    current = current.next

current.next = new_node

def display(self):

    current = self.head

    values = []

    while current is not None:

        values.append(str(current.data))

        current = current.next

    print("-> ".join(values) if values else "The list is empty.")

if __name__ == "__main__":

    linked_list = LinkedList()

    linked_list.insert(10)

    linked_list.insert(20)

    linked_list.insert(30)

    print("Linked list:")

    linked_list.display()

```

Output:

```

PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/t11.3.py"
Linked list:
10 -> 20 -> 30
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#Q4

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
    pass
```

Prompt: generate a code on BST with insert and in-order traversal methods.

Code:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
class BST:
```

```
    def __init__(self):
```

```
        self.root = None
```

```
    def insert(self, data):
```

```
        """Insert a value into the tree."""
```

```
        self.root = self._insert(self.root, data)
```

```
    def _insert(self, node, data):
```

```
        if node is None:
```

```
            return Node(data)
```

```
        if data < node.data:
```

```
            node.left = self._insert(node.left, data)
```

```
        elif data > node.data:
```

```
            node.right = self._insert(node.right, data)
```

```
        return node
```

```
    def in_order(self):
```

```
        values = []
```

```

        self._in_order(self.root, values)

    return values

def _in_order(self, node, values):

    if node is not None:

        self._in_order(node.left, values)

        values.append(node.data)

        self._in_order(node.right, values)

if __name__ == "__main__":

    bst = BST()

    for value in (50, 30, 70, 20, 40, 60, 80):

        bst.insert(value)

    print("In-order traversal:", bst.in_order())

```

output:

```

PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/t11.4.py"
In-order traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\chait\OneDrive\Desktop\AIAC>

```

#Q5

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```

class PriorityQueue:

    pass

```

Prompt: generate a code to implement a priority queue using Python's heapq module.

Code:

```

import heapq

```

```
class PriorityQueue:

    def __init__(self):

        self._items = []


    def insert(self, item):

        """Add an item to the priority queue."""

        heapq.heappush(self._items, item)


    def remove(self):

        """Remove and return the item with the highest priority."""

        return heapq.heappop(self._items)


    def peek(self):

        """Return the highest-priority item without removing it."""

        return self._items[0]


    def is_empty(self):

        return len(self._items) == 0


if __name__ == "__main__":

    queue = PriorityQueue()

    for item in (5, 1, 3, 2, 4):

        queue.insert(item)


    print("Priority order:", end="")

    while not queue.is_empty():
```

```
print(queue.remove(),end="")
```

output:

```
PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/t11.5.py"
Priority order: 1 2 3 4 5
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

#Q6

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

class DequeDS:

prompt: generate a code to implement a double-ended queue using

collections.deque.

code:

```
from collections import deque
```

```
class DoubleEndedQueue:
```

```
    """A double-ended queue (deque)."""
```

```
    def __init__(self):
```

```
        """Create an empty deque."""
```

```
        self._items = deque()
```

```
    def add_front(self, item):
```

```
        """Add an item to the front of the deque."""
```

```
        self._items.appendleft(item)
```

```
    def add_rear(self, item):
```

```
        """Add an item to the rear of the deque."""
```

```
self._items.append(item)
```

```
def remove_front(self):
```

```
    """Remove and return the front item, or None if empty."""
```

```
    if self.is_empty():
```

```
        return None
```

```
    return self._items.popleft()
```

```
def remove_rear(self):
```

```
    """Remove and return the rear item, or None if empty."""
```

```
    if self.is_empty():
```

```
        return None
```

```
    return self._items.pop()
```

```
def is_empty(self):
```

```
    """Return True when the deque has no items."""
```

```
    return len(self._items) == 0
```

```
if __name__ == "__main__":
```

```
    queue = DoubleEndedQueue()
```

```
    queue.add_rear(10)
```

```
    queue.add_rear(20)
```

```
    queue.add_front(5)
```

```
    print("Deque after adding 10, 20 at the rear and 5 at the front:", queue._items)
```

```
    print("Removed from front:", queue.remove_front())
```

```
print("Removed from rear:", queue.remove_rear())
```

```
print("Deque after removals:", queue._items)
```

```
print("Is deque empty?", queue.is_empty())
```

output:

```
PS C:\Users\chait\OneDrive\Desktop\AIAC> & C:/Users/chait/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/chait/OneDrive/Desktop/AIAC/AIAC LAB 7.1/AIAC LAB 4/AIAC LAB-11.1/t6.py"
Deque after adding 10, 20 at the rear and 5 at the front: deque([5, 10, 20])
Removed from front: 5
Removed from rear: 20
Deque after removals: deque([10])
Is deque empty? False
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

Q7

Task: Real-Time Application Challenge– Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System

that handles:

1. Student Attendance Tracking– Daily log of students

entering/exiting the campus.

2. Event Registration System– Manage participants in events with

quick search and removal.

3. Library Book Borrowing– Keep track of available books and their

due dates.

4. Bus Scheduling System– Maintain bus routes and stop

connections.

5. Cafeteria Order Queue– Serve students in the order they arrive.

Student Task:

For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

Justify your choice in 2– 3 sentences per feature.

Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

A table mapping feature → chosen data structure → justification.

A functional Python program implementing the chosen feature with comments and docstrings.

Prompt: Create a simple Python Event Registration System using a Hash Table (dictionary). Include student registration, searching, deleting, and displaying records using Student ID. Add comments, docstrings, and sample output. Keep the code beginner-friendly and menu-driven.

Code:

```
"""Simple event registration system backed by a hash table (dictionary)."""
```

```
registrations = {}
```

```
def register():
```

```
    """Register or update a student's event registration."""
```

```
    student_id = input("Enter Student ID: ").strip()
```

```
if not student_id:

    print("Student ID cannot be empty.")

    return


name = input("Enter Student Name:").strip()
event = input("Enter Event Name:").strip()
registrations[student_id] = [name, event]

print("Registration successful!")
```

```
def search():

    """Search for a student using Student ID."""

    student_id = input("Enter Student ID:").strip()

    if student_id in registrations:

        name, event = registrations[student_id]

        print("Name:", name)

        print("Event:", event)

    else:

        print("Student not found.")
```

```
def remove():

    """Remove a student's registration."""

    student_id = input("Enter Student ID:").strip()

    if student_id in registrations:

        del registrations[student_id]

        print("Registration removed!")

    else:
```

```
print("Student not found.")
```

```
def display():
```

```
    """Display all registered students."""
```

```
    if not registrations:
```

```
        print("No registrations found.")
```

```
    return
```

```
for student_id, (name, event) in registrations.items():
```

```
    print(student_id, name, event)
```

```
def main():
```

```
    """Run the event registration menu."""
```

```
    while True:
```

```
        print("\n1. Register")
```

```
        print("2. Search")
```

```
        print("3. Remove")
```

```
        print("4. Display")
```

```
        print("5. Exit")
```

```
    choice = input("Enter choice:").strip()
```

```
    if choice == "1":
```

```
        register()
```

```
    elif choice == "2":
```

```
        search()
```

```
    elif choice == "3":
```

```

        remove()

elif choice == "4":

    display()

elif choice == "5":

    print("Thank you!")

    break

else:

    print("Invalid choice")

if __name__ == "__main__":

    main()

```

output:

```

1. Register
2. Search
3. Remove
4. Display
5. Exit
Enter choice: 1
Enter Student ID: 101
Enter Student Name: chaithra
Enter Event Name: coding contest
Registration successful!

1. Register
2. Search
3. Remove
4. Display
5. Exit
Enter choice: 1
Enter Student ID: 102
Enter Student Name: anjali
Enter Event Name: hackathon
Registration successful!

1. Register
2. Search
3. Remove
4. Display
5. Exit
Enter choice: 4
101 chaithra coding contest
102 anjali hackathon

1. Register
2. Search
3. Remove
4. Display
5. Exit

Enter choice: 2
Enter Student ID: 101
Name: chaithra
Event: coding contest

1. Register
2. Search
3. Remove
4. Display
5. Exit
Enter choice: 3
Enter Student ID: 102
Registration removed!

1. Register
2. Search
3. Remove
4. Display
5. Exit
Enter choice: 5
Thank you!

```

Q8

Task: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Deque

Justify your choice in 2– 3 sentences per feature.

Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

A table mapping feature → chosen data structure → justification.

A functional Python program implementing the chosen feature with comments and docstrings.

Prompt: Create a simple Python Smart E-Commerce System using suitable data structures. Implement shopping cart, order processing, top-selling products, product search, and delivery planning. Add comments, docstrings, and sample output.

Code:

```
# Smart E-Commerce System
```

```
# Data Structure: Deque
```

```
from collections import deque
```

```
cart = deque()
```

```
def add_product(product):
```

```
    """Add a product to the shopping cart."""
```

```
    cart.append(product)
```

```
    print(product, "added to cart.")
```

```
def remove_product():
```

```
    """Remove the last product from the shopping cart."""
```

```
    if cart:
```

```
    product = cart.pop()

    print(product, "removed from cart.")

else:

    print("Cart is empty.")
```

```
def display_cart():

    """Display all products in the cart."""

    print("Shopping Cart:", list(cart))
```

```
# Main program
```

```
add_product("Laptop")
```

```
add_product("Headphones")
```

```
add_product("Mouse")
```

```
display_cart()
```

```
remove_product()
```

```
display_cart()
```

output:

```
C:\Desktop\AIAC>python AIAC.py
Laptop added to cart.
Headphones added to cart.
Mouse added to cart.
Shopping Cart: ['Laptop', 'Headphones', 'Mouse']
Mouse removed from cart.
Shopping Cart: ['Laptop', 'Headphones']
PS C:\Users\chait\OneDrive\Desktop\AIAC>
```

